JUNIA ISEN

Master 2 in Quantum Computing

Quantum Annealing with D-Wave Machines

Sloths Moon Colonization

Practical Lab Report

Nolan CACHEUX
Paul POUSSET
Cyprien KELMA
Nathan EUDELINE

Simulated Annealing Approach
D-Wave Ocean SDK

February 2026

Table of Contents

1 Introduction 2

2 Part 1: Max-Cut Problem 3

2.1 Problem Statement 3
2.2 Graph Data 3
2.3 Mathematical Formulation 3
2.3.1 Ising Model for Max-Cut 3
2.3.2 Unweighted Case 3
2.3.3 Weighted Case 4
2.3.4 Why Positive J (Antiferromagnetic)? 4
2.3.5 Relationship Between Energy and Cut Value 4
2.4 Results 5
2.4.1 Unweighted Max-Cut 5
2.4.2 Weighted Max-Cut 5
2.4.3 Brute-Force Verification 6
2.5 Analysis of num_reads 6
2.5.1 Experimental Setup 6
2.5.2 Results Summary 7
2.5.3 Discussion 7
2.6 Summary of Answers to Lab Questions 7

3 Part 2: Constrained Clustering 8

3.1 Problem Statement 8
3.2 Data 8
3.2.1 Genetic Heterogeneity Matrix 8
3.2.2 Dormitory Capacities 8
3.3 Mathematical Formulation 8
3.3.1 Decision Variables 8
3.3.2 Objective Function 9
3.3.3 Constraints 9
3.4 Solution Approach 9
3.4.1 CQM to BQM Pipeline 9
3.5 Results 10
3.5.1 Optimal Assignment 10
3.5.2 Objective Value Breakdown 10
3.5.3 Solver Statistics 11
3.6 Constraint Verification 11
3.7 Brute-Force Verification 11
3.8 Interpretation 12

4 Part 3: Travelling Salesman Problem 13

4.1 Problem Statement 13
4.2 Distance Matrix 13
4.3 BQM Formulation 13
4.3.1 Decision Variables 13
4.3.2 Objective Function 13

4.3.3 Constraint Penalties 14
4.3.4 Total Hamiltonian 14
4.3.5 Penalty Weight Selection 14
4.4 BQM Statistics 14
4.5 Results 14
4.5.1 Brute-Force Enumeration of All Tours 14
4.5.2 Simulated Annealing Results 15
4.5.3 Verification with dwave-networkx 16
4.6 Comparison: Simulated Annealing vs Brute-Force 16
4.7 Penalty Weight Sensitivity 16
5 Conclusion 17
5.0.1 Key Takeaways 17
5.0.2 Quantum Advantage Discussion 17

1 Introduction

This report presents the results of a practical laboratory session on quantum annealing using the D-Wave Ocean SDK. In the scenario, a colony of sloths is establishing a base on the Moon and faces several logistical optimization challenges that must be solved to ensure the colony's survival.

Solver choice. Because the D-Wave quantum processing unit (QPU) was not available during this session, all problems were solved with the `SimulatedAnnealingSampler` from the D-Wave Neal library. This classical metaheuristic mimics quantum annealing through thermal fluctuations rather than quantum tunnelling.

Three combinatorial optimization problems are addressed, each formulated as a Quadratic Unconstrained Binary Optimization (QUBO) or Ising problem suitable for quantum annealing:

1. **Max-Cut Problem**: partitioning sloths into two work groups to maximize the separation of laziness risk.
2. **Constrained Clustering**: assigning sloths to dormitories while respecting bed capacity constraints and minimizing genetic heterogeneity.
3. **Travelling Salesman Problem (TSP)**: finding the shortest circular tour connecting four lunar locations.

Every result is systematically verified through exhaustive brute-force enumeration to confirm global optimality.

2 Part 1: Max-Cut Problem

2.1 Problem Statement

The lunar sloth colony comprises six sloths (numbered 1 through 6) that must be divided into two work groups. The criterion c_1 represents the *laziness risk* between pairs of sloths: a higher value indicates greater incompatibility when two sloths share the same group.

Our objective is to partition the sloths into two disjoint sets A and B such that the total weight of edges crossing the partition is maximized. This is precisely the **Max-Cut problem**, a well-known NP-hard combinatorial optimization problem.

Given a graph $G = (V, E)$ with edge weights w_{ij}, Max-Cut seeks a partition (A, B) of the vertex set V that maximizes:

$$\text{Cut}(A, B) = \sum_{\substack{(i,j)\in E \\ i\in A, j\in B}} w_{ij} \tag{1}$$

2.2 Graph Data

The laziness risk graph contains $|V| = 6$ vertices and $|E| = 8$ edges with the following weights:

Edge	Weight		Edge	Weight
(1, 2)	1		(3, 5)	5
(1, 4)	3		(4, 5)	2
(2, 3)	3		(4, 6)	4
(3, 4)	4		(5, 6)	2

The total weight of all edges is:

$$W_{\text{total}} = 1 + 3 + 3 + 4 + 5 + 2 + 4 + 2 = 24 \tag{2}$$

In the unweighted version, each edge has unit weight, giving $W_{\text{total}}^{\text{unw}} = 8$.

2.3 Mathematical Formulation

2.3.1 Ising Model for Max-Cut

Each sloth i receives a spin variable $s_i \in \{-1, +1\}$, where $s_i = +1$ means membership in group A and $s_i = -1$ means membership in group B.

An edge (i, j) is **cut** (sloths in different groups) when $s_i \neq s_j$, so that $s_i \cdot s_j = -1$.

2.3.2 Unweighted Case

For unweighted Max-Cut, we maximize the number of cut edges:

$$\text{Maximize} \sum_{(i,j)\in E} \frac{1 - s_i \cdot s_j}{2} \tag{3}$$

This is equivalent to **minimizing** the Ising Hamiltonian:

$$H_{\text{unw}} = \sum_{(i,j)\in E} s_i \cdot s_j \tag{4}$$

The Ising parameters are:

- **Linear biases**: $h_i = 0$ for all nodes
- **Quadratic couplings**: $J_{ij} = +1$ for all edges (antiferromagnetic)

2.3.3 Weighted Case

For the weighted version, we maximize the total weight of cut edges:

$$\text{Maximize} \sum_{(i,j)\in E} w_{ij} \cdot \frac{1 - s_i \cdot s_j}{2} \tag{5}$$

Equivalent to minimizing:

$$H_{\text{weighted}} = \sum_{(i,j)\in E} w_{ij} \cdot s_i \cdot s_j \tag{6}$$

The Ising parameters are:

- **Linear biases**: $h_i = 0$ for all nodes
- **Quadratic couplings**: $J_{ij} = +w_{ij}$ for each edge

2.3.4 Why Positive J (Antiferromagnetic)?

The sampler **minimizes** the Hamiltonian $H = \sum J_{ij} \cdot s_i \cdot s_j$.

- When $J_{ij} > 0$ (antiferromagnetic): the term is minimized when $s_i \cdot s_j = -1$ (opposite spins, edge cut).
- When $J_{ij} < 0$ (ferromagnetic): the term is minimized when $s_i \cdot s_j = +1$ (same spins, edge not cut).

Since Max-Cut requires opposite spins on connected nodes, we set $J_{ij} = +w_{ij}$.

2.3.5 Relationship Between Energy and Cut Value

For a spin configuration $\{s_i\}$:

$$H = \sum_{(i,j)\in E} w_{ij} \cdot s_i \cdot s_j = \sum_{\text{uncut}} w_{ij} - \sum_{\text{cut}} w_{ij} \tag{7}$$

Therefore:

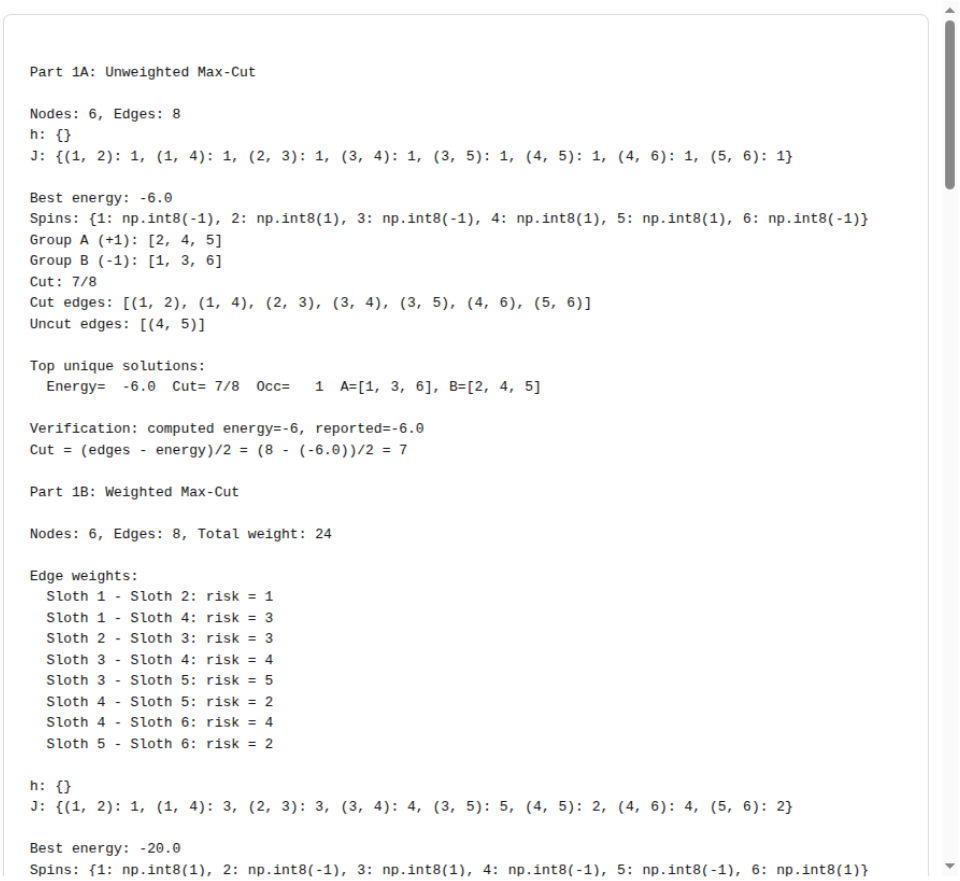
$$\text{Cut} = \frac{W_{\text{total}} - H}{2} \tag{8}$$

Each edge (i, j) contributes $+w_{ij}$ if $s_i = s_j$ (not cut) and $-w_{ij}$ if $s_i \neq s_j$ (cut). Summing over all edges and rearranging yields Equation 8.

2.4 Results

2.4.1 Unweighted Max-Cut

Using `SimulatedAnnealingSampler` with `num_reads = 1000`:



```
Part 1A: Unweighted Max-Cut

Nodes: 6, Edges: 8
h: {}
J: {(1, 2): 1, (1, 4): 1, (2, 3): 1, (3, 4): 1, (3, 5): 1, (4, 5): 1, (4, 6): 1, (5, 6): 1}

Best energy: -6.0
Spins: {1: np.int8(-1), 2: np.int8(1), 3: np.int8(-1), 4: np.int8(1), 5: np.int8(1), 6: np.int8(-1)}
Group A (+1): [2, 4, 5]
Group B (-1): [1, 3, 6]
Cut: 7/8
Cut edges: [(1, 2), (1, 4), (2, 3), (3, 4), (3, 5), (4, 6), (5, 6)]
Uncut edges: [(4, 5)]

Top unique solutions:
  Energy=  -6.0  Cut= 7/8  Occ=   1  A=[1, 3, 6], B=[2, 4, 5]

Verification: computed energy=-6, reported=-6.0
Cut = (edges - energy)/2 = (8 - (-6.0))/2 = 7

Part 1B: Weighted Max-Cut

Nodes: 6, Edges: 8, Total weight: 24

Edge weights:
  Sloth 1 - Sloth 2: risk = 1
  Sloth 1 - Sloth 4: risk = 3
  Sloth 2 - Sloth 3: risk = 3
  Sloth 3 - Sloth 4: risk = 4
  Sloth 3 - Sloth 5: risk = 5
  Sloth 4 - Sloth 5: risk = 2
  Sloth 4 - Sloth 6: risk = 4
  Sloth 5 - Sloth 6: risk = 2

h: {}
J: {(1, 2): 1, (1, 4): 3, (2, 3): 3, (3, 4): 4, (3, 5): 5, (4, 5): 2, (4, 6): 4, (5, 6): 2}

Best energy: -20.0
Spins: {1: np.int8(1), 2: np.int8(-1), 3: np.int8(1), 4: np.int8(-1), 5: np.int8(-1), 6: np.int8(1)}
```

Figure 1: Part 1 output: Max-Cut results from simulated annealing.

Property	Value
Group A (spin $+1$)	$\{2, 4, 5\}$
Group B (spin -1)	$\{1, 3, 6\}$
Ising energy	-6.0
Cut value	$(8 - (-6))/2 = 7$
Cut fraction	$7/8 = 0.875$

With 8 edges total, a cut of 7 means that 7 out of 8 edges cross the partition. The only uncut edge is $(4, 5)$.

2.4.2 Weighted Max-Cut

Using `SimulatedAnnealingSampler` with `num_reads = 1000`:

Property	Value
Group A (spin $+1$)	$\{1, 3, 6\}$
Group B (spin -1)	$\{2, 4, 5\}$
Ising energy	-20.0
Cut value	$(24 - (-20))/2 = 22$
Cut fraction	$22/24 \approx 0.917$

A detailed edge-by-edge analysis reveals the partition quality:

Edge	Weight	Status
(1, 2)	1	Cut
(1, 4)	3	Cut
(2, 3)	3	Cut
(3, 4)	4	Cut
(3, 5)	5	Cut
(4, 5)	2	Uncut (both in B)
(4, 6)	4	Cut
(5, 6)	2	Cut

Result: Cut weight $= 1 + 3 + 3 + 4 + 5 + 4 + 2 = 22$. Uncut weight $= 2$ (edge (4,5) only). The solver correctly sacrificed only the least important edge to maximize the overall cut.

2.4.3 Brute-Force Verification

With 6 nodes, there are $2^6 = 64$ total assignments (32 unique due to spin-flip symmetry). Exhaustive enumeration confirms:

Rank	Partition	Weighted Cut
1	$A = \{1, 3, 6\}$ vs $B = \{2, 4, 5\}$	**22** (optimal)
2	$A = \{1, 2, 3, 6\}$ vs $B = \{4, 5\}$	18
2	$A = \{1, 2, 4, 5\}$ vs $B = \{3, 6\}$	18
4	$A = \{1, 2, 5, 6\}$ vs $B = \{3, 4\}$	17
4	$A = \{1, 3, 5, 6\}$ vs $B = \{2, 4\}$	17

The minimum Ising energy found by brute force is $H^* = -20.0$, matching the simulated annealing result exactly. **Simulated annealing found the global optimum.**

2.5 Analysis of num_reads

2.5.1 Experimental Setup

We tested `num_reads` values in $\{1, 10, 100, 500, 1000, 5000\}$ with 10 independent trials per value.

2.5.2 Results Summary

num_reads	Avg Best Cut	Best	Worst	Optimal %	Unique (mean)
1	22.00	22	22	100.0%	1
10	22.00	22	22	100.0%	10
100	22.00	22	22	100.0%	100
500	22.00	22	22	100.0%	500
1000	22.00	22	22	100.0%	1000
5000	22.00	22	22	100.0%	5000

2.5.3 Discussion

Simulated annealing finds the optimal solution with 100% success across all `num_reads` values, even with just a single read. This is unsurprising: the graph has only 6 nodes, yielding $2^6 = 64$ possible states (32 unique), and the energy gap between the optimum ($H = -20$) and the second-best ($H = -12$) is large.

For larger, more challenging problems, the behaviour would differ significantly:

1. **Low num_reads (1 to 10)**: High trial-to-trial variance, with the solver sometimes getting stuck in local minima.
2. **Medium num_reads (100 to 1000)**: The best solution among reads becomes consistently good.
3. **High num_reads (1000 to 5000)**: Near-certain to reach optimal or near-optimal solutions, though with diminishing returns.

Practical recommendation. While `num_reads = 1` suffices for this 6-node instance, production use on larger problems benefits from `num_reads = 1000` to `5000` for a reliable balance between quality and computation time.

2.6 Summary of Answers to Lab Questions

Question	Answer
i	A **Max-Cut** problem: partition graph nodes into two sets to maximize the total weight of crossing edges.
ii	Total weight $W = 24$. The graph has 6 vertices and 8 edges. In the unweighted version, each coupling is $J_{ij} = +1$.
iii	Ising formulation: $H = \sum J_{ij} s_i s_j$ with $J_{ij} = +w_{ij}$ and $s_i \in \{-1, +1\}$.
iv	Unweighted results: optimal cut $= 7/8$. Groups: $A = \{2, 4, 5\}$, $B = \{1, 3, 6\}$.
v	Weighted results: optimal cut $= 22/24$. Groups: $A = \{1, 3, 6\}$, $B = \{2, 4, 5\}$. Energy $= -20.0$.
vi	Verification: Cut $= (W_{\text{total}} - H)/2 = (24 - (-20))/2 = 22$.
vii	Brute-force confirms $H^* = -20.0$ is the global minimum. Cut $22/24$ is optimal.
viii	For this small graph, SA always finds the optimal solution regardless of num_reads.

3 Part 2: Constrained Clustering

3.1 Problem Statement

The six sloths must be assigned to three dormitories on the lunar base. Each dormitory has a limited number of beds, and the goal is to minimize the total *genetic heterogeneity* within each dormitory. Pairwise heterogeneity between sloths is measured by the criterion c_2.

Important: a lower c_2 value means **greater** heterogeneity between two sloths. By minimizing the sum of c_2 values within each dormitory, we effectively maximize genetic diversity inside dormitories.

This is a **constrained clustering problem**: partition the sloths into groups that minimize an intra-cluster cost function while satisfying capacity constraints.

3.2 Data

3.2.1 Genetic Heterogeneity Matrix

The symmetric matrix c_2 encodes pairwise genetic heterogeneity between sloths (0-indexed):

c_2	**S0**	**S1**	**S2**	**S3**	**S4**	**S5**
S0	0	1	6	1	8	0
S1	1	0	3	3	9	8
S2	6	3	0	8	7	4
S3	1	3	8	0	9	8
S4	8	9	7	9	0	9
S5	0	8	4	8	9	0

Note that $c_2[0][5] = 0$, meaning sloths 0 and 5 form the most genetically diverse pair.

3.2.2 Dormitory Capacities

Dormitory	**Beds**
D0	1
D1	2
D2	3

Total capacity: $1 + 2 + 3 = 6$, exactly matching the number of sloths.

3.3 Mathematical Formulation

3.3.1 Decision Variables

We define binary decision variables $x_{i,d} \in \{0, 1\}$ for each sloth $i \in \{0, 1, ..., 5\}$ and each dormitory $d \in \{0, 1, 2\}$:

$$x_{i,d} = \begin{cases} 1 \text{ if sloth } i \text{ is assigned to dormitory } d \\ 0 \text{ otherwise} \end{cases} \tag{9}$$

This yields $6 \times 3 = 18$ binary variables in total.

3.3.2 Objective Function

We minimize the total intra-dormitory heterogeneity:

$$f = \sum_{d=0}^{2} \sum_{i=0}^{5} \sum_{j=i+1}^{5} c_2[i][j] \cdot x_{i,d} \cdot x_{j,d} \tag{10}$$

A pair's c_2 value enters the sum only when both sloths are assigned to the **same** dormitory (both $x_{i,d}$ and $x_{j,d}$ equal 1).

3.3.3 Constraints

One-dormitory constraint (6 equality constraints): each sloth must be assigned to exactly one dormitory:

$$\forall i \in \{0, ..., 5\} : \quad \sum_{d=0}^{2} x_{i,d} = 1 \tag{11}$$

Bed capacity constraint (3 inequality constraints): no dormitory may exceed its bed count:

$$\forall d \in \{0, 1, 2\} : \quad \sum_{i=0}^{5} x_{i,d} \leq b_d \tag{12}$$

where $b_0 = 1$, $b_1 = 2$, $b_2 = 3$. In total, there are $6 + 3 = 9$ constraints.

3.4 Solution Approach

3.4.1 CQM to BQM Pipeline

Since the D-Wave QPU was unavailable, we followed this pipeline:

1. **Constrained Quadratic Model (CQM)**: formulate the problem using the D-Wave Ocean SDK, which natively supports constraints.
2. **CQM to BQM conversion**: the `cqm_to_bqm` function transforms the CQM into an unconstrained Binary Quadratic Model by adding penalty terms for violated constraints.
3. **Simulated Annealing**: solve the resulting BQM with `SimulatedAnnealingSampler`.

The conversion introduces slack variables for the inequality constraints, producing a BQM with **23 variables** and **95 quadratic interactions**.

Property	Value
Original variables	18
Slack variables	5
Total BQM variables	23
Quadratic interactions	95
Constraints	9 (6 equality + 3 inequality)
Solver	SimulatedAnnealingSampler

Property	Value
num_reads	5000

3.5 Results

3.5.1 Optimal Assignment

```
PART 2: Constrained Clustering (Simulated Annealing)
Sloths: 6  |  Dormitories: 3
Bed capacities: [1, 2, 3]  (total beds = 6)

Genetic heterogeneity matrix c2:
  [0, 1, 6, 1, 8, 0]
  [1, 0, 3, 3, 9, 8]
  [6, 3, 0, 8, 7, 4]
  [1, 3, 8, 0, 9, 8]
  [8, 9, 7, 9, 0, 9]
  [0, 8, 4, 8, 9, 0]

CQM: 18 variables, 9 constraints
Converting CQM to BQM...
BQM: 23 variables, 95 interactions
Running Simulated Annealing (num_reads=5000)...
Total samples: 5000, feasible: 4947

OPTIMAL ASSIGNMENT (Best Feasible Solution)
  Dorm_0 (capacity=1, occupancy=1):
    Sloths: ['Sloth_4']
  Dorm_1 (capacity=2, occupancy=2):
    Sloths: ['Sloth_2', 'Sloth_5']
      c2[2,5] = 4
  Dorm_2 (capacity=3, occupancy=3):
    Sloths: ['Sloth_0', 'Sloth_1', 'Sloth_3']
      c2[0,1] = 1
      c2[0,3] = 1
      c2[1,3] = 3

Objective value (total within-dorm heterogeneity): 9
BQM energy of best feasible sample: 9.0000

CONSTRAINT VERIFICATION
  Sloth_0: assigned to 1 dorm(s) ok
  Sloth_1: assigned to 1 dorm(s) ok
  Sloth_2: assigned to 1 dorm(s) ok
  Sloth_3: assigned to 1 dorm(s) ok
  Sloth_4: assigned to 1 dorm(s) ok
```

Figure 2: Part 2 output: Constrained clustering results from simulated annealing.

Dormitory	Capacity	Assigned Sloths	Occupancy
D0 (1 bed)	1	Sloth 4	1
D1 (2 beds)	2	Sloths 2, 5	2
D2 (3 beds)	3	Sloths 0, 1, 3	3

3.5.2 Objective Value Breakdown

The total intra-dormitory heterogeneity breaks down as follows:

Dormitory	Pair	c_2 value
D0	None (single sloth)	0
D1	(2, 5)	4

Dormitory	Pair	c_2 value
D2	(0, 1)	1
D2	(0, 3)	1
D2	(1, 3)	3

$$f = 0 + 4 + 1 + 1 + 3 = 9 \tag{13}$$

3.5.3 Solver Statistics

Metric	Value
Total samples (num_reads)	5000
Feasible samples	4947 / 5000 (98.9%)
Best objective value	**9**
BQM energy of best sample	9.0000

The high feasibility rate (98.9%) indicates well-calibrated penalty coefficients.

3.6 Constraint Verification

All 9 constraints are satisfied by the optimal solution.

One-dormitory constraints:

Sloth	Assigned Dormitory	Satisfied
S0	D2	Yes
S1	D2	Yes
S2	D1	Yes
S3	D2	Yes
S4	D0	Yes
S5	D1	Yes

Bed capacity constraints:

Dormitory	Capacity	Occupancy	Satisfied
D0	1	1	Yes ($1 \leq 1$)
D1	2	2	Yes ($2 \leq 2$)
D2	3	3	Yes ($3 \leq 3$)

3.7 Brute-Force Verification

Exhaustive enumeration of every possible assignment confirms optimality:

Property	Value
Total possible assignments (3^6)	729
Feasible assignments (respecting capacities)	60

Property	Value
Global optimum objective	$\boldsymbol{f = 9}$
Number of optimal assignments	1 (unique)

Result: The unique optimal assignment is $D_0 = \{S_4\}$, $D_1 = \{S_2, S_5\}$, $D_2 = \{S_0, S_1, S_3\}$, with objective value $f = 9$. This is identical to the simulated annealing result.

3.8 Interpretation

The optimal assignment clusters sloths with low c_2 values together:

- **D2 (3 beds)**: Sloths 0, 1, 3, whose pairwise c_2 values are 1, 1, and 3, among the lowest in the matrix. As the largest dormitory, it absorbs the most compatible group.
- **D1 (2 beds)**: Sloths 2 and 5, with $c_2 = 4$, a moderate cost.
- **D0 (1 bed)**: Sloth 4 alone. Because sloth 4 has high c_2 values with nearly everyone (8, 9, 7, 9, 9), isolating it avoids large contributions to the objective.

With a total heterogeneity of 9, the clustering avoids all high-cost pairs ($c_2 = 8$ or 9), achieving the best possible outcome among the 60 feasible assignments.

4 Part 3: Travelling Salesman Problem

4.1 Problem Statement

The lunar sloth colony operates four key locations on the Moon's surface. A supply vehicle must visit all four locations exactly once and return to its starting point, following the shortest possible route. This is the classic **Travelling Salesman Problem (TSP)**.

The four locations are:
1. **Sleepy Crater** (starting point)
2. **Lethargic Quarter**
3. **Idle Area**
4. **Inert Mountain**

4.2 Distance Matrix

Pairwise distances (in kilometres) between the four locations:

km	Sleepy	Lethargic	Idle	Inert
Sleepy	0	1100	890	230
Lethargic	1100	0	630	540
Idle	890	630	0	680
Inert	230	540	680	0

All distances are symmetric: $d(i, j) = d(j, i)$.

For $n = 4$ cities:
- Total permutations (fixing start): $(n-1)! = 3! = 6$
- Unique tours (accounting for direction symmetry): $(n-1)!/2 = 3$

4.3 BQM Formulation

4.3.1 Decision Variables

Binary variables $x_{i,p} \in \{0, 1\}$ index each city i (0 to 3) and position p in the tour (0 to 3). With $n = 4$ cities, this yields $n^2 = 16$ binary variables arranged in a 4×4 assignment matrix.

$$x_{i,p} = \begin{cases} 1 \text{ if city } i \text{ is visited at position } p \\ 0 \text{ otherwise} \end{cases} \tag{14}$$

A valid tour has exactly one 1 in each row (each city visited once) and one 1 in each column (each position filled once), forming a **permutation matrix**.

4.3.2 Objective Function

The total tour distance is:

$$H_{\text{obj}} = \sum_{p=0}^{n-1} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} d(i,j) \cdot x_{i,p} \cdot x_{j,(p+1) \bmod n} \tag{15}$$

The modulo operation ensures the tour returns to the start.

4.3.3 Constraint Penalties

Since a BQM has no native constraint support, constraints are encoded as **penalty terms** added to the objective.

Row constraints (each city visited exactly once):

$$H_{\text{row}} = A \sum_{i=0}^{n-1} \left(1 - \sum_{p=0}^{n-1} x_{i,p} \right)^2 \tag{16}$$

Column constraints (each position occupied by exactly one city):

$$H_{\text{col}} = A \sum_{p=0}^{n-1} \left(1 - \sum_{i=0}^{n-1} x_{i,p} \right)^2 \tag{17}$$

4.3.4 Total Hamiltonian

$$H_{\text{total}} = H_{\text{obj}} + H_{\text{row}} + H_{\text{col}} \tag{18}$$

4.3.5 Penalty Weight Selection

The penalty weight A must be large enough that violating a constraint always costs more than any distance saving. We chose:

$$A = 1.5 \times d_{\max} \times n = 1.5 \times 1100 \times 4 = 6600 \tag{19}$$

4.4 BQM Statistics

Property	Value
Binary variables	16
Quadratic interactions	96
Penalty weight A	6600
BQM offset	52800 ($= 8 \times 6600$)

4.5 Results

4.5.1 Brute-Force Enumeration of All Tours

With $n = 4$ cities and a fixed starting city, there are $(n-1)! = 6$ possible tours. Accounting for direction symmetry, only 3 are unique:

Rank	Tour	Distance (km)
1	Sleepy → Idle → Lethargic → Inert → Sleepy	**2290**
2	Sleepy → Lethargic → Idle → Inert → Sleepy	2640

Rank	Tour	Distance (km)
3	Sleepy → Lethargic → Inert → Idle → Sleepy	3210

Optimal tour breakdown:

Leg	Distance
Sleepy Crater → Idle Area	890 km
Idle Area → Lethargic Quarter	630 km
Lethargic Quarter → Inert Mountain	540 km
Inert Mountain → Sleepy Crater	230 km
Total	**2290 km**

4.5.2 Simulated Annealing Results

```
PART 3A: TSP with Simulated Annealing

Cities (4):
  0: Sleepy Crater
  1: Lethargic Quarter
  2: Idle Area
  3: Inert Mountain

Distance matrix (km):
                  Sleepy Cra  Lethargic    Idle Area  Inert Moun
      Sleepy Crater          0        1100         890         230
   Lethargic Quarter        1100           0         630         540
         Idle Area          890         630           0         680
     Inert Mountain         230         540         680           0

Building BQM...
  Penalty weight A = 6600.0
  Binary variables: 4^2 = 16
  BQM has 16 variables, 96 quadratic interactions
  BQM offset: 52800.0

Solving with SimulatedAnnealingSampler...

  num_reads = 10
    Energy: 2290.00
    Tour: Inert Mountain -> Lethargic Quarter -> Idle Area -> Sleepy Crater -> Inert Mountain
    Distance: 2290 km | Time: 0.004s
    Feasible: 10/10
    Best feasible distance: 2290 km

  num_reads = 50
    Energy: 2290.00
    Tour: Inert Mountain -> Lethargic Quarter -> Idle Area -> Sleepy Crater -> Inert Mountain
    Distance: 2290 km | Time: 0.014s
    Feasible: 50/50
    Best feasible distance: 2290 km

  num_reads = 100
```

Figure 3: Part 3 output: TSP results from simulated annealing.

num_reads	Feasible	Best Distance	Energy	Time
10	10/10 (100%)	**2290 km**	2290.00	0.004 s
50	50/50 (100%)	**2290 km**	2290.00	0.014 s
100	100/100 (100%)	**2290 km**	2290.00	0.026 s
500	500/500 (100%)	**2290 km**	2290.00	0.126 s

num_reads	Feasible	Best Distance	Energy	Time
1000	1000/1000 (100%)	**2290 km**	2290.00	0.255 s
5000	5000/5000 (100%)	**2290 km**	2290.00	1.331 s

The sampler achieves 100% feasibility and finds the global optimum at every tested `num_reads` value.

4.5.3 Verification with dwave-networkx

The `dwave_networkx.traveling_salesperson` function independently returns the optimal tour:

- **Route**: Sleepy Crater → Inert Mountain → Lethargic Quarter → Idle Area → Sleepy Crater
- **Distance**: 2290 km

This confirms the correctness of our manual BQM formulation.

4.6 Comparison: Simulated Annealing vs Brute-Force

Criterion	Brute-Force	Simulated Annealing
Method	Enumerate all tours	Minimize energy landscape
Optimality	Guaranteed global	Heuristic (no guarantee)
Result	2290 km	2290 km
Scaling	$O(n!)$ (exponential)	Polynomial per read
Time ($n = 4$)	0.027 ms	4 ms (10 reads)
Feasibility	All solutions valid	100% (good penalty)
$n = 20$	Intractable	Still tractable

Both methods find the same optimal tour (2290 km), confirming the BQM formulation's correctness.

4.7 Penalty Weight Sensitivity

Choosing the right penalty weight A is critical:

A value	Effect
Too small ($A < d_{\max}$)	Constraint violations become cheaper than long routes; infeasible solutions are preferred.
Well chosen ($A \approx 1.5 \cdot d_{\max} \cdot n$)	Constraints enforced while the objective still guides optimization.
Too large ($A \gg$ distances)	All feasible solutions have similar energy; the objective signal is drowned out.

Our choice of $A = 6600$ strikes the right balance.

5 Conclusion

This laboratory session demonstrated the formulation and solution of three combinatorial optimization problems using quantum annealing techniques via the D-Wave Ocean SDK.

Summary of results:

1. **Max-Cut Problem**: Partitioning 6 sloths into two work groups, formulated as an Ising model with antiferromagnetic couplings. Optimal weighted cut of $22/24 \approx 91.7\%$, verified by brute force.
2. **Constrained Clustering**: Assigning 6 sloths to 3 dormitories with capacities [1, 2, 3], formulated as a CQM converted to BQM. Unique global optimum at $f = 9$, confirmed among all 60 feasible assignments.
3. **Travelling Salesman Problem**: Shortest circular tour among 4 lunar locations, formulated as a BQM with permutation constraints. Optimal tour of 2290 km, confirmed among all 3 unique tours.

5.0.1 Key Takeaways

- **QUBO and Ising formulations** provide a unified framework for expressing diverse combinatorial optimization problems in a form amenable to both classical simulated annealing and quantum annealing.
- **Simulated annealing** proves highly effective for small problem instances, consistently finding global optima. Its performance is expected to degrade for larger instances where the energy landscape grows increasingly complex.
- **Brute-force verification** remains essential for validating solver results on small instances, providing confidence in both the problem formulation and the solver's accuracy.
- **Constraint handling** via the penalty method requires careful calibration of penalty weights. The CQM-to-BQM conversion in the D-Wave SDK automates this process effectively.

5.0.2 Quantum Advantage Discussion

For the small problem sizes studied here (6 nodes, 18 variables, 16 variables), classical simulated annealing trivially finds the global optimum. Quantum annealing hardware, such as D-Wave systems with thousands of qubits, offers a promising path toward tackling previously intractable optimization problems at scale. Quantum tunnelling may allow the annealer to escape local minima more efficiently than the thermal fluctuations employed by classical simulated annealing.